

Weather Data Compression and Clustering

Bashar Albaalbaki

Coursera Project

IBM Machine Learning Certificate Series

Course #4: Unsupervised Machine Learning

Final Project

1. Introduction

In this project we will look at collected weather data for years 2023 and 2024 for several cities across the globe. The collected data includes a timestamp, dry bulb temperature, relative humidity, wind speed and direction, cloudiness, rain and a few other weather data. In total 13 data points are collected for each hour for each city in the two-year period. Next, we will try to compress the data using various dimensionality reductions reviewed in the course including PCA, kernel PCA. Finally, we will attempt to cluster the mixed data of cities and evaluate how the data will be clustered for example we will investigate whether the clustered weather data is classified based on the location.

Machine learning methods tried in the project:

- 1) Linear PCA dimensionality reduction algorithm
- 2) Kernel PCA dimensionality reduction algorithm
- 3) MDS dimensionality reduction algorithm
- 4) KNN clustering
- 5) DBSCAN clustering
- 6) Mean shift clustering



Austin-TX



Calgary-AB



Lincoln-NE



Lisbon-Portugal



Los Angeles-CA



Paris-France

Figure 1. Austin-TX, Calgary-AB, Lincoln-NE, Lisbon-Portugal, Los Angeles-CA, Paris-France.

2. Data Collection

Source of data: Here the weather data is taken from Open-Metro API that provides hourly weather data for free for non-commercial purposes. The data that was gathered for this project are:

1. Timestamp
2. Temperature, °C
3. Relative Humidity %
4. Precipitation, mm
5. Rain, mm
6. Showers, mm
7. Snow Fall, cm

8. Dew Point, °C
9. Wind Speed, km/hr
10. Wind Direction, °
11. Wind Gusts, km/hr
12. Cloud Cover. %
13. Surface Pressure, hPa

OpenMetro gives data in the json format which here we changed into the csv format using json and pandas modules in Python. In the notebook the csv file for each city is used to populate a pandas DataFrame for each city. For example, for Austin, TX, a sample of collected data is presented in figure 2 below.

	timestamp	temperature_2m	relative_humidity_2m	precipitation	rain	showers	snowfall	snow_depth	dew_point_2m	wind_speed_10m	wind_direction_10m	wind_gusts_10m	cloud_cover	surface_pressure
2794	2023-04-27 10:00:00	7.2	27	0.0	0.0	0.0	0.0	0.01	-10.6	13.8	261	51.5	7	611.2
14786	2024-09-08 02:00:00	11.1	62	0.0	0.0	0.0	0.0	0.00	4.0	3.6	84	16.9	50	621.6
5971	2023-09-06 19:00:00	2.7	85	0.0	0.0	0.0	0.0	0.00	0.5	5.3	242	14.4	25	612.0
7625	2023-11-14 17:00:00	-4.6	61	0.0	0.0	0.0	0.0	0.01	-11.0	8.7	246	34.6	12	606.6
3174	2023-05-13 06:00:00	5.6	29	0.0	0.0	0.0	0.0	0.03	-11.2	6.7	344	29.5	100	618.6

Figure 2. A sample of data taken from Open-Metro API for Austin, TX.

Next step is to check if there is any non-numbers (NaN) in the data, using pandas isnull() function , with its results being shown in figure 3, we find that except for Lisbon data, all data are in fact numeric. For Lisbon, whole of NaN data are in the “snow_depth” column which is fully is filled with NaN for all the rows. Therefore, “snow_depth” column for Lisbon is filled with the number zero.

```
City: Austin, Number of rows with a NaN = 0
City: Calgary, Number of rows with a NaN = 0
City: Lincoln_NB, Number of rows with a NaN = 0
City: Lisbon, Number of rows with a NaN = 17544
      Number of rows with a NaN in snow_depth column = 17544
      Number of rows with a NaN in other column = 0
City: LosAngeles, Number of rows with a NaN = 0
City: Paris, Number of rows with a NaN = 0
```

Figure 3. Checking NaN data in each city DataFrame file.

Next, we combine data file’s corresponding to different cities into a single pandas DataFrame. We will keep the city name in a column so in further steps below we will able to assess our clustering results match to cities. A sample of data is shown in figure 4.

df_cities.sample(10)																
	timestamp	temperature_2m	relative_humidity_2m	precipitation	rain	showers	snowfall	snow_depth	dew_point_2m	wind_speed_10m	wind_direction_10m	wind_gusts_10m	cloud_cover	surface_pressure	city	
52258	2023-12-17 10:00:00	14.9	57	0.0	0.0	0.0	0.00	0.00	6.6	37.8	72	49.7	53	1034.1	Lisbon	
72387	2024-04-03 03:00:00	14.8	83	0.1	0.1	0.0	0.00	0.00	11.9	14.3	28	27.4	100	1010.3	LosAngeles	
71539	2024-02-27 19:00:00	-1.3	90	0.0	0.0	0.0	0.00	0.01	-2.7	6.5	93	13.3	100	1025.0	LosAngeles	
24902	2023-11-03 14:00:00	-9.8	69	0.0	0.0	0.0	0.00	0.02	-14.4	7.8	103	15.1	100	923.0	Calgary	
25994	2023-12-19 02:00:00	-25.2	64	0.0	0.0	0.0	0.00	0.04	-30.0	16.1	310	33.5	87	937.6	Calgary	
54427	2024-03-16 19:00:00	15.4	86	0.0	0.0	0.0	0.00	0.00	13.1	33.1	292	44.3	49	1022.7	Lisbon	
67183	2023-08-30 07:00:00	27.6	52	0.0	0.0	0.0	0.00	0.00	16.9	8.7	38	21.6	12	1009.0	LosAngeles	
239	2023-01-10 23:00:00	-10.4	27	0.0	0.0	0.0	0.00	0.00	-25.8	9.9	251	30.6	15	596.5	Austin	
19482	2023-03-22 18:00:00	-11.9	76	0.2	0.0	0.0	0.14	0.13	-15.3	17.6	329	37.8	100	921.3	Calgary	
62309	2023-02-08 05:00:00	8.8	66	0.0	0.0	0.0	0.00	0.00	2.7	17.4	68	34.2	100	1025.7	LosAngeles	

Figure 4. Ten samples of the final DataFrame

Next, we will two new features to the data: hour of the day and day of the year are extracted from the timestamp as two new numeric columns.

⚠ ⚠ Since for each city we are only considering two-year of weather data, for each city the day of the year appears only twice. Note that all the cities considered in this project are in the northern hemisphere, so the annual weather cycle is almost the same for all cities.

Encoding city names: Using sklearn's LabelEncoder(), the name of the cities is transformed into new 6 columns, shown in figure 5.

```
le = LabelEncoder()
city_labels = le.fit_transform(df_cities["city"].values)
for label in le.classes_:
    print(f"{label}: label = {le.transform([label])[0]}")

Austin: label = 0
Calgary: label = 1
Lincoln_NB: label = 2
Lisbon: label = 3
LosAngeles: label = 4
Paris: label = 5
```

Figure 5. Transferring the city names into 6 binary columns.

Finally, after removing the columns for city name and timestamp, we end up with a data with 96480 samples of weather data and with having 21 numerical columns.

Data size: 96480 x 21

3. Feature engineering

Next step is to example the simple correlations between various features. At first glance it is realized that the feature "showers" is 0 for all the rows, as shown in figure 6. Thus, this column is removed.

```
[20]: data[["showers"]].describe()
```

```
[20]:
```

	showers
count	96480.0
mean	0.0
std	0.0
min	0.0
25%	0.0
50%	0.0
75%	0.0
max	0.0

Figure 6. Statistics of the column "showers"

Now we can see the correlation among the columns. The heat map of such correlations is shown in figure 7.

```
data_corr_matrix = data.corr()  
plt.figure(figsize=(12, 12)) # Adjust figure size as needed  
sns.heatmap(data_corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
```

Figure 7a. The code to create the correlation heat-map of data

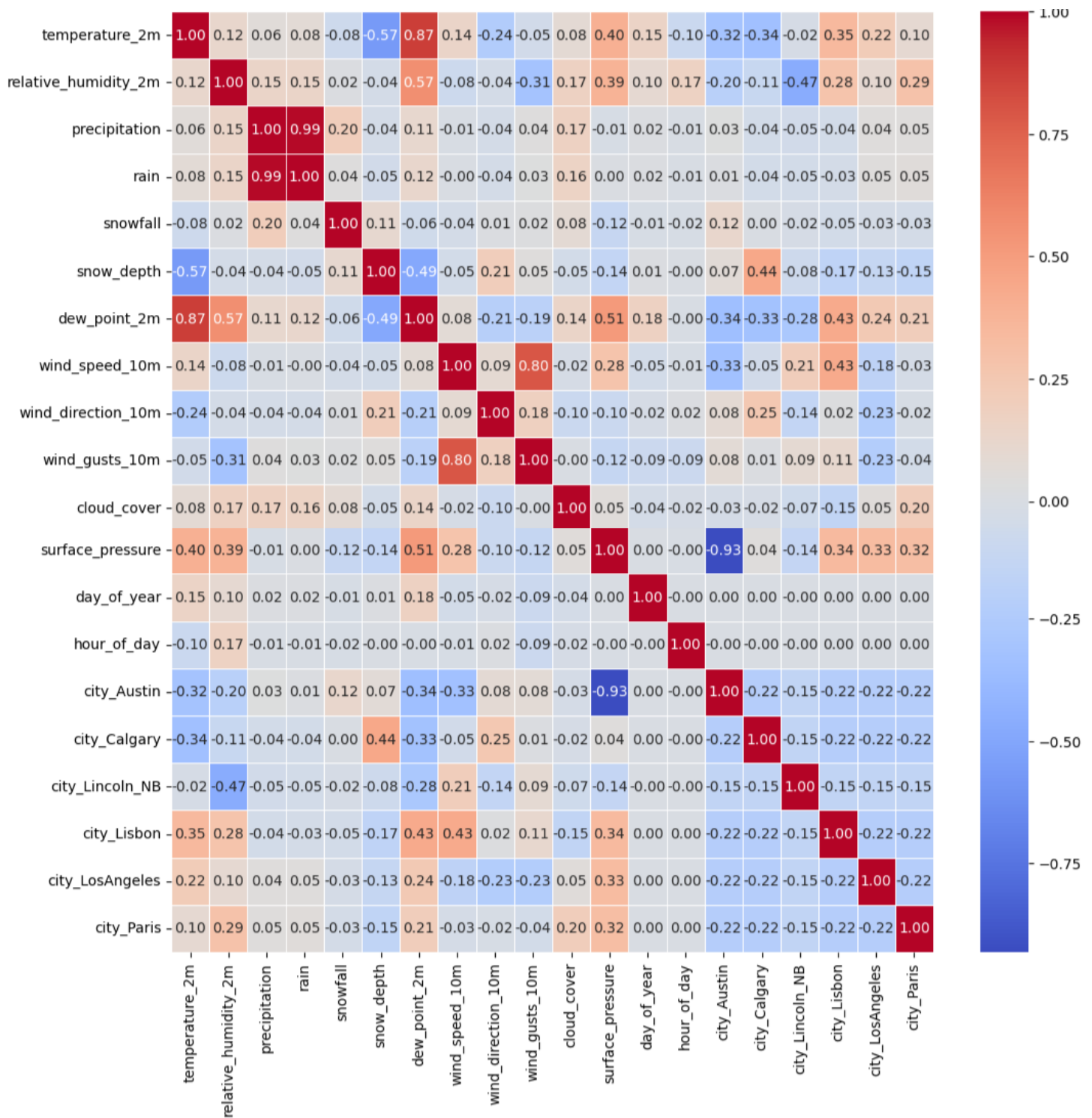


Figure 7b. The heat-map of data

As can be observed from the correlation heat-map in figure 7b, columns "rain" and "participation" are correlated strongly and thus one of them, "rain" is removed.

Removing the column "rain": Data size: 96480 x 19

4. Dimensionality Reduction

First, we copy all data except 6 binary columns corresponding city names to have a pure numerical, and call it X. Next, we split X into training and test portions and prepare standard and min-max scalers that is shown in figure 9.

```
X = data.drop(columns=encoder.get_feature_names_out())
X.shape

(96480, 13)
```

Figure 8. Copying data to X except for binary label columns for city names.

```
indices = np.arange(X.shape[0])
X_train, X_test, indices_train, indices_test = train_test_split(X.values, indices, test_size = 0.2, random_state = 42)

ss = StandardScaler()
X_train_scaled = ss.fit_transform(X_train)

mms = MinMaxScaler()
X_train_mmscaled = mms.fit_transform(X_train)
```

Figure 9. Processing of X before dimensionality reduction

4.1. Principal component analysis (PCA)

First, we start we an ambitious goal of reducing dimensionality of data to two dimensions. If successful, we can have nice visualization of data. We will assess the success of dimensionality reduction in this exercise and all subsequent ones by trying to compress and decompress the test data and then measure the closeness of the recovered data with the original test data. Figure 10 shows the codes and subsequently the assessment for using PCA for 2D dimensionality reduction. Note that the input to the PCA is the standard scaled training X.

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_train_scaled)
X_pca.shape

(77184, 2)

Xhat_test = ss.inverse_transform(pca.inverse_transform(pca.transform(ss.transform(X_test))))
r2 = r2_score(y_true=X_test, y_pred=Xhat_test)
r2

0.37587289898796405
```

Figure 10. Dimensionality reduction to 2D by PCA

As can be seen from figure 10 the R2 score of the 2D PCA is only 38%. In this work we set the success criterion to 95% R2 score for the recovery of the compressed data. Then, although the 2D-PCA is now deemed unsuccessful, still, let's see how its visualization will look which is shown in figure 11.

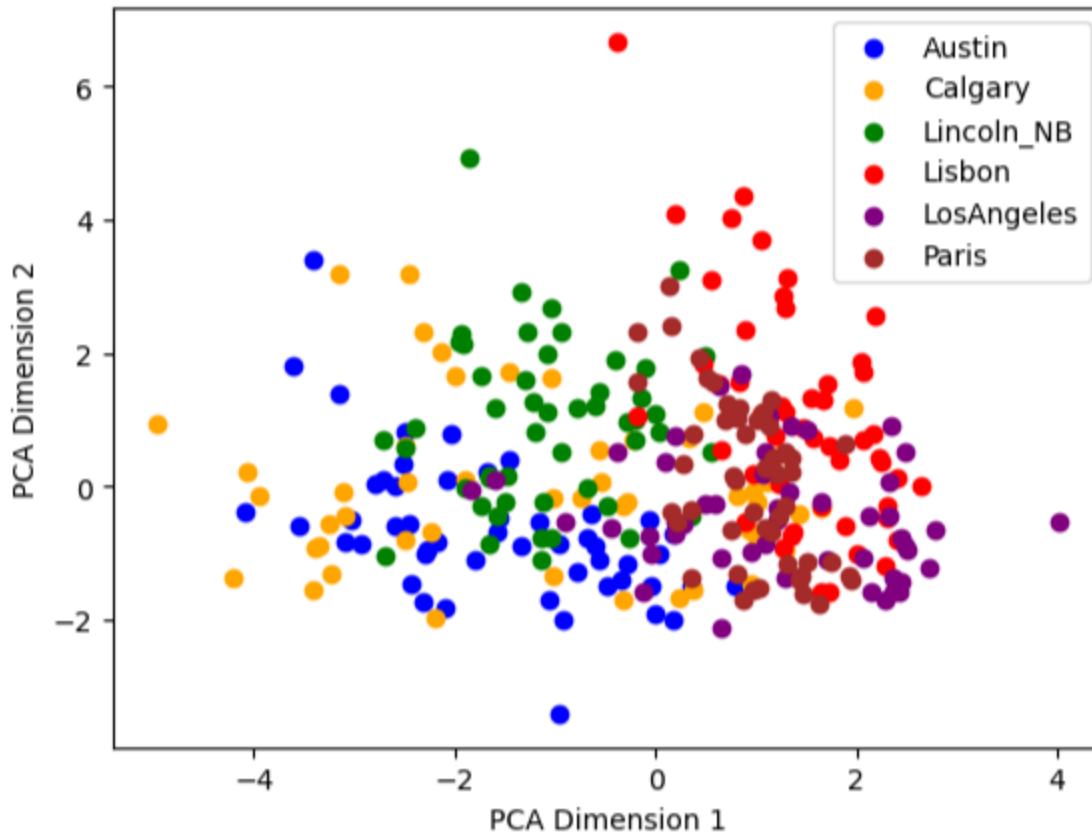


Figure 11. Visualization of outcome of 2D-PCA

Considering figure 11, it's difficult to find any clustering of data at least by visual inspection. In the section we will examine clustering using appropriate clustering algorithms, but for now and in this section, we will only focus on dimensionality reduction. It's interesting to see that when data is scaled, for examples cities of Austin, TX and Calgary, AB although belonging to two apparently different climates, but overlap in figure 11.

One more visualization for 2D PCA, we will attempt to create city-pairs and examine all possible pairs and see how their compressed climate data are compared. Such a comparison is shown in figure 12. Looking at figure 12 we observe that:

1. Ausin weather data overlaps with Calgary and Lincoln data but for other cities of Los Angeles, Paris and Lisbon the overlap is minimal.
2. For Calgary and Lincoln, they follow a similar pattern with Austin.
3. Paris, Lisbon and Los Angeles data overlap.

Thus, considering both figures 11 and 12 we can suggest visually the following clustering of cities:

Cluster 1: Austin, Calgary, Lincoln

Cluster 2: Los Angeles, Paris, Lisbon

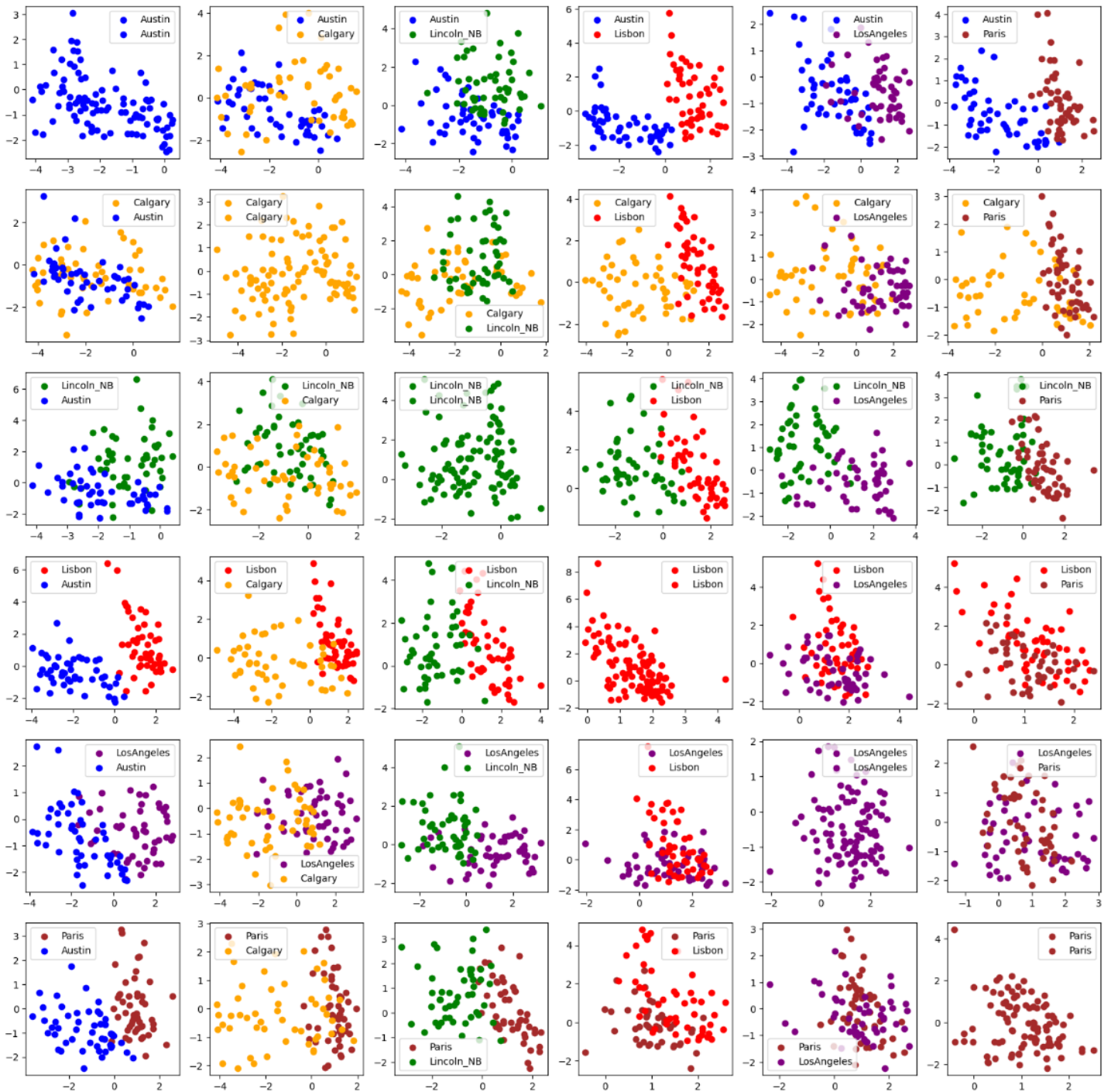


Figure 12. Comparing individual city pairs. Here horizontal and vertical axes correspond to the first and second PCA axis.

Next, we will attempt the PCA with dimensionality more than 2 and observe the recovery R2 score for each dimensionality reduction which is shown in figure 13.

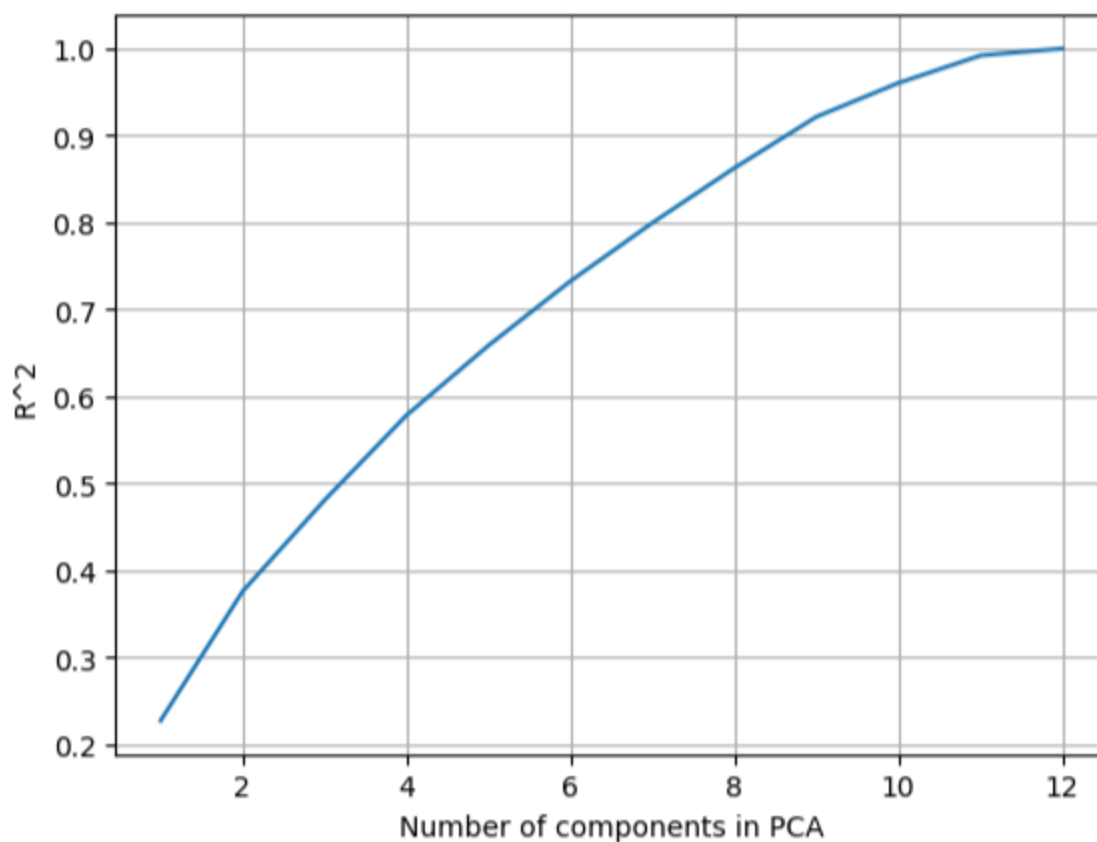


Figure 13. Extending PCA to more than 2D

```
n_components_arr = np.arange(1,X_train_scaled.shape[1])
r2_arr = []
for n_components in n_components_arr:
    pca_n = PCA(n_components=n_components)
    pca_n.fit(X_train_scaled)
    Xhat_test = ss.inverse_transform(pca_n.inverse_transform(pca_n.transform(ss.transform(X_test))))
    r2 = r2_score(y_true=X_test, y_pred=Xhat_test)
    r2_arr.append(r2)
    print(f"n_components = {n_components}, R^2 = {r2}")
plt.plot(n_components_arr, r2_arr)
plt.xlabel("Number of components in PCA")
plt.ylabel("R^2")
plt.grid(True)
plt.show()
```

Figure 14 the code for figure 13

The number shown in figure 13 are:

n_components = 1, R^2 = 0.22747548739274848
n_components = 2, R^2 = 0.3758728989879635
n_components = 3, R^2 = 0.48067363211998776
n_components = 4, R^2 = 0.578237529995545
n_components = 5, R^2 = 0.6583591509916579

n_components = 6, $R^2 = 0.7323453354000182$
n_components = 7, $R^2 = 0.7992734805660369$
n_components = 8, $R^2 = 0.8622902775169496$
n_components = 9, $R^2 = 0.921004296908582$
n_components = 10, $R^2 = 0.959852569624304$
n_components = 11, $R^2 = 0.9913621837658677$
n_components = 12, $R^2 = 0.999344761785111$

Thus, we can use PCA to compress data from having 13 components to 10 with recovery R^2 score of 96%.

4.2. Kernel PCA

First, to make the problem tractable we will only use 5% of the standard scaled data. As shown in figure 15 Kernel PCA was not able to recover the original data with a score higher than 80%.

```
indices = np.random.choice(len(X_train_scaled), size=len(X_train_scaled)//20, replace=False)
# replace=False causes the indices to be unique and non-repeated.
X_train_scaled_compact = X_train_scaled[indices, :]
```

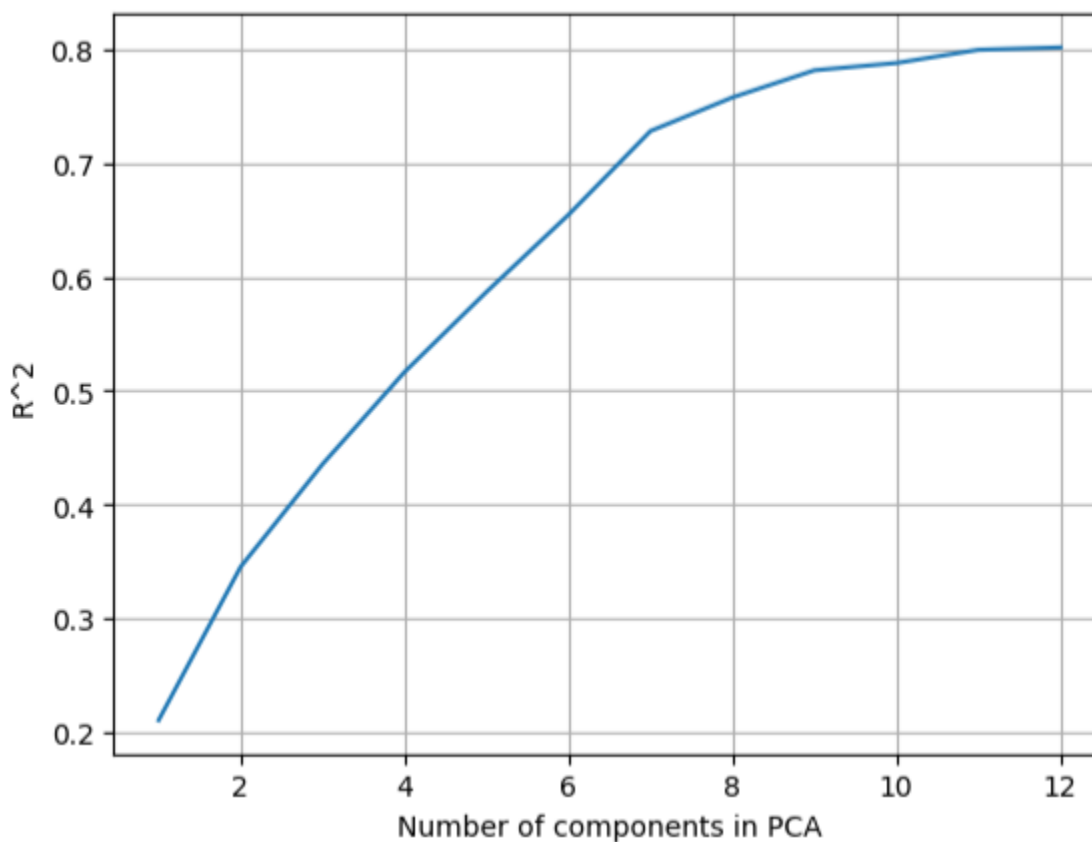


Figure 15. Kernel PCA of type "RBF"

```

n_components_arr = np.arange(1,X_train_scaled.shape[1])
r2_arr = []
for n_components in n_components_arr:
    kpca_n = KernelPCA(n_components=n_components,
                       kernel='rbf',
                       gamma=None,
                       fit_inverse_transform=True,
                       alpha=0.1,
                       )
    kpca_n.fit(X_train_scaled_compact)
    Xhat_test = ss.inverse_transform(kpca_n.inverse_transform(kpca_n.transform(ss.transform(X_test))))
    r2 = r2_score(y_true=X_test, y_pred=Xhat_test)
    r2_arr.append(r2)
    print(f"n_components = {n_components}, R^2 = {r2}")
plt.plot(n_components_arr, r2_arr)
plt.xlabel("Number of components in PCA")
plt.ylabel("R^2")
plt.grid(True)
plt.show()

```

Figure 16. Code for figure 15.

Next, we take advantage of the data compression by the 10D PCA, obtained in the previous section to see if applying the Kernel PCA on the compressed data by PCA will have any effect. The outcome of that process is shown in figure 17.

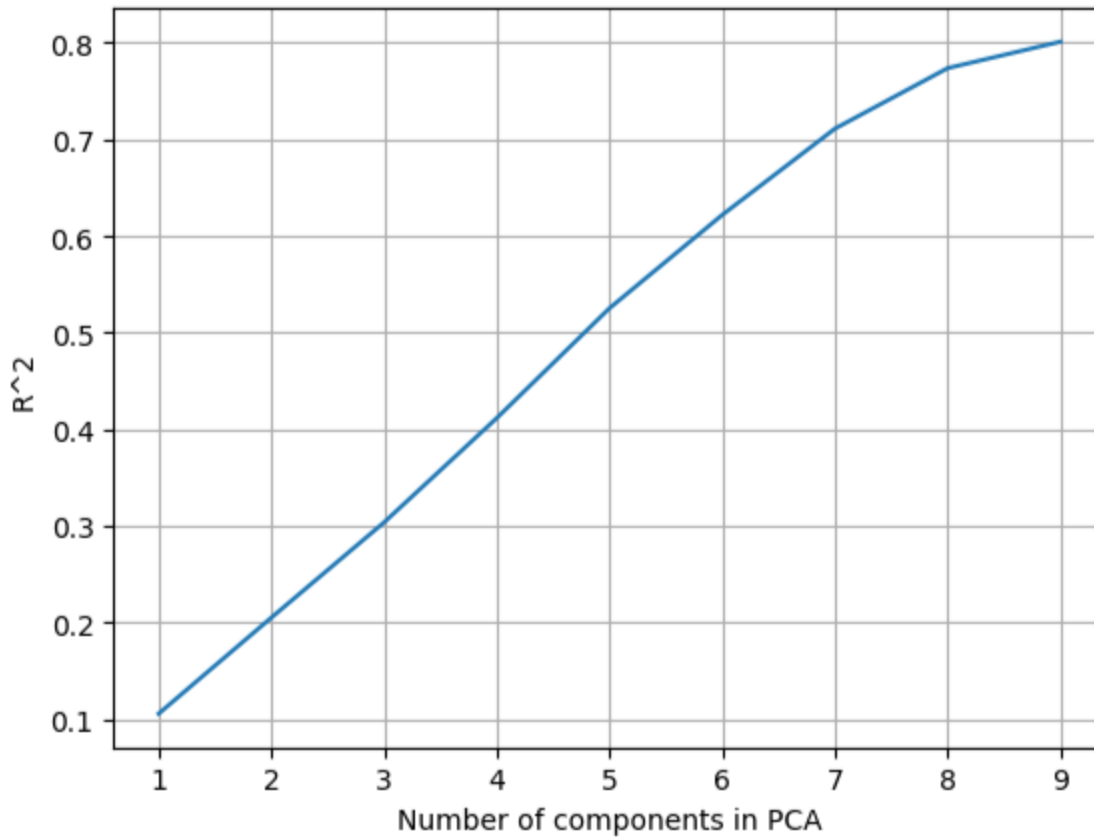


Figure 17. Applying “RBF” Kernel PCA on data already compressed by 10D Linea PCA

```
X_train2, X_test2 = train_test_split(X_pca10, test_size=0.95, random_state=123, stratify=df_cities["city"].values)
test_indices = np.random.choice(len(X_test2), size=len(X_test2)//20, replace=False)
ss2 = StandardScaler()
X_train2_scaled = ss2.fit_transform(X_train2)

n_components_arr = np.arange(1, X_train2_scaled.shape[1])
r2_arr = []
for n_components in n_components_arr:
    kpca_n = KernelPCA(n_components=n_components, kernel='rbf', gamma=None, fit_inverse_transform=True, alpha=0.1)
    kpca_n.fit(X_train2_scaled)
    Xhat_test2 = ss2.inverse_transform(kpca_n.inverse_transform(kpca_n.transform(ss2.transform(X_test2[test_indices,:]))))
    r2 = r2_score(y_true=X_test2[test_indices,:], y_pred=Xhat_test2)
    r2_arr.append(r2)
    print(f"n_components = {n_components}, R^2 = {r2}")
plt.plot(n_components_arr, r2_arr)
plt.xlabel("Number of components in PCA")
plt.ylabel("R^2")
plt.grid(True)
plt.show()
```

Figure 18. Code for figure 17.

In conclusion we see that Kernel PCA is less efficient than PCA for the data of interest in this work.

4.3. Multi-Dimensional Scaling (MDS)

Like PCA and Kernel PCA we apply the MDS dimensionality reduction algorithm. We apply MDS only for visualization as MDS does not have an inverse method (at least for sklearn package) and thus we cannot assess its effectiveness. Moreover, to make the data tractable we randomly selected 5000 samples from the training data to train the MDS. Next, we present the dimensionality reduction by MDS by presenting the data for cities in a pair-wise manner, shown in figure 19.

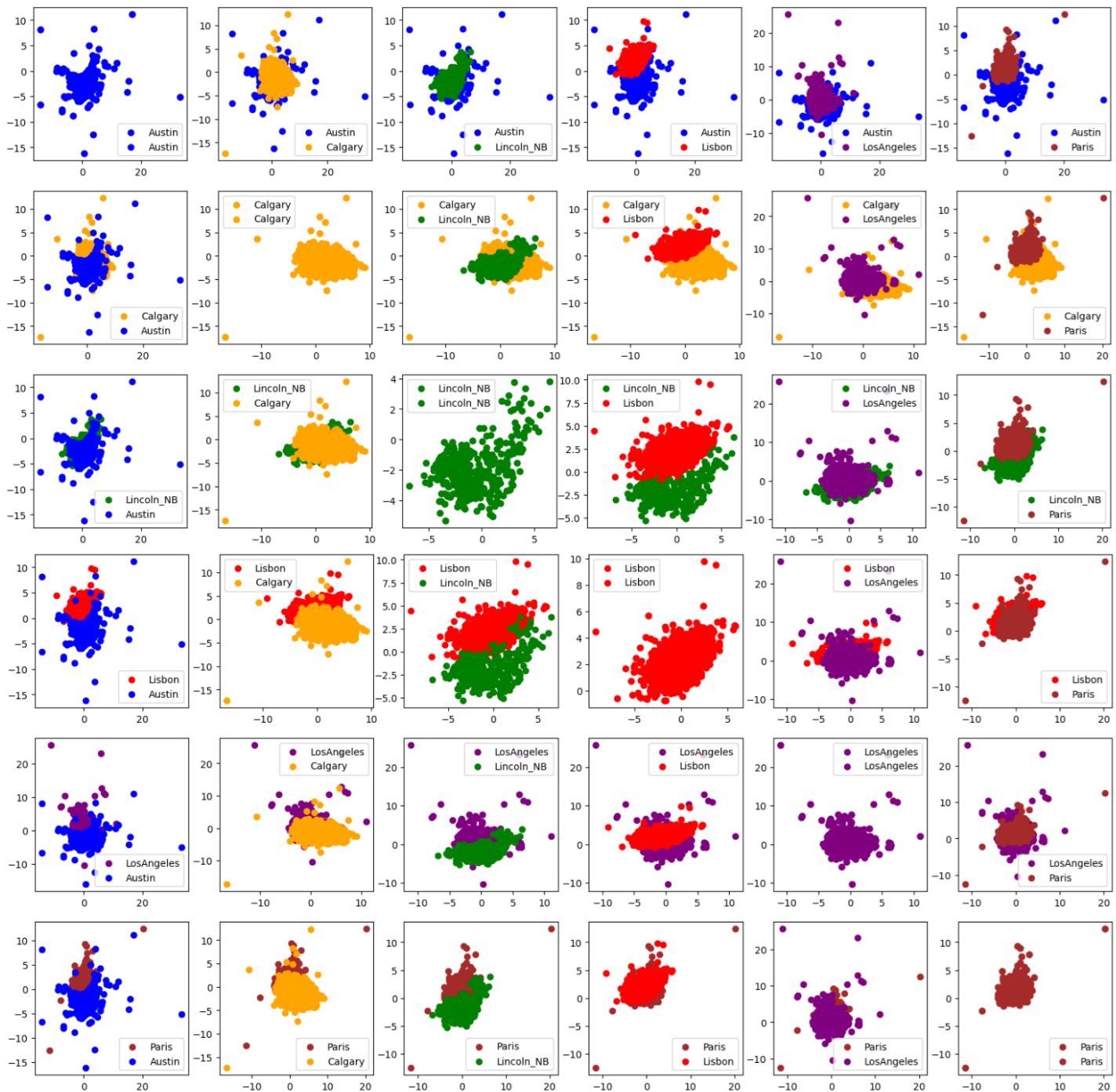


Figure 19. MDS outcome using only 5000 of the training data


```

indices = np.random.choice(len(data), size=5000, replace=False) # replace=False causes the indices to be unique and non-repeated.
data_scaled_compact = ss.transform(X.values[indices, :])

t0 = time.time()
n_components = 2
print(f"n_components = {n_components}")
embedding = MDS(n_components=n_components, n_init=1, verbose=1, random_state=42)
X_embedding = embedding.fit_transform(data_scaled_compact)
t1 = time.time()
print(f"---DONE, t = {t1-t0} sec ----")
print(f"X_embedding.shape = {X_embedding.shape}")

n_components = 2
---DONE, t = 172.08329892158508 sec ----
X_embedding.shape = (5000, 2)

df_mds = pd.DataFrame()
df_mds["mds_x1"] = X_embedding[:,0]
df_mds["mds_x2"] = X_embedding[:,1]
df_mds["city"] = df_cities['city'].values[indices]
print(f"df_mds sample = {df_mds.sample(20)}")
for city in cities:
    df_city = df_mds[df_mds["city"] == city]
    print(f"\nNumber of entries for {city} = {len(df_city)}")
fig, ax = plt.subplots(nrows=n_cities, ncols=n_cities, figsize=(20,20))
for i, city1 in enumerate(cities):
    for j, city2 in enumerate(cities):
        for city in [city1, city2]:
            df_city = df_mds[df_mds["city"] == city]
            # df_city = df_city.sample(50)
            ax[i][j].scatter(df_city["mds_x1"].values, df_city["mds_x2"].values, label=city, color=colors[city])
            ax[i][j].legend()
plt.show()

```

Figure 20. Code for figure 19.

4.4 Conclusion for dimensionality reduction

As discussed above linear PCA performed superior to Kernel PCA. Note that although not shown here but I have tried other types of Kernel PCA available in sklearn and the "rbf" type seemed to be performing better. Also, by looking at figure 19 and comparing it with figure 12 it seems that linear PCA also outperforms MDS.

5. Clustering weather data

In this section we wish to cluster the weather data, still without feeding the city labels, or the 6 binary columns of city columns and examine if the weather data is clustered according to location or else.

5.1.KNN Clustering

First, we apply KNN clustering to all the training data. However, unlike the previous section where the standard scaling was used here the min-max scaling is applied on the training data before feeding them into KNN clustering function. Figure 21 presents the code for KNN clustering.

```

n_cities = len(cities)
print(f"n_cities = {n_cities}")
kmeans = KMeans(n_clusters=n_cities, random_state=0)
kmeans.fit(X_train_mmscaled)

```

Figure 21. Code for clustering

Now we will attempt to compare clustering found by the KNN clustering and the clustering based on the solely city label. However, we do this comparison not using the training data but using the test data that were not part of the data fed to KNN function. For the comparison, first the contingency matrix is presented in figure 22. Note that contingency matrix is simply a table that shows the how data of one cluster of a clustering method is distributed among the clusters of another clustering approach. We will also compute Adjusted Rand matrix (ARI) and Normalized Mutual Information (NMI). For both ARI and NMI, score of 1.0 indicates perfect alignment of clustering methods. The results of are:

Adjusted Rand Index (ARI) = 0.15625048350832024

Normalized Mutual Information (NMI) = 0.27500512111164327

The values of ARI and NMI indicate low level of similarity between clustering found by KNN and cities. Figure 22 reveals the following facts

1. Cluster-3 includes almost only data points from Austin whereas Austin data almost equally is distributed between clusters 3 and 5.
2. Cluster 5 includes almost only data from Austin, Lincoln and Calgary. Recall that in the previous section PCA we found indications that these three cities belong to a same cluster.

	city	cluster-0	cluster-1	cluster-2	cluster-3	cluster-4	cluster-5
0	Austin	0	0	0	1823	0	1645
1	Calgary	384	877	737	0	483	1054
2	Lincoln_NB	211	290	406	2	159	719
3	Lisbon	1112	654	550	0	1199	0
4	LosAngeles	817	1174	755	0	783	19
5	Paris	579	1321	1031	0	512	0

Figure 22. The contingency matrix for comparing clustering based on KNN and one based on city label for the test data. Here cluster-0 to cluster-5 are clusters found by KNN clustering.

```

ari = adjusted_rand_score(df_knn["city_code"] , df_knn["cluster_label"])
print(f"Adjusted Rand Index (ARI) = {ari}")
nmi = normalized_mutual_info_score(df_knn["city_code"] , df_knn["cluster_label"])
print(f"Normalized Mutual Information (NMI) = {nmi}")
cmat = contingency_matrix(labels_true=df_knn["city_code"] , labels_pred=df_knn["cluster_label"])
cmat_df = pd.DataFrame(columns=["city"] + [f"cluster-{i}" for i in range(6)])
cmat_df["city"] = cities
cmat_df.iloc[:,1:] = cmat
print(f"Contingency Matrix:")
cmat_df

```

Figure 23. Code for figure 22.

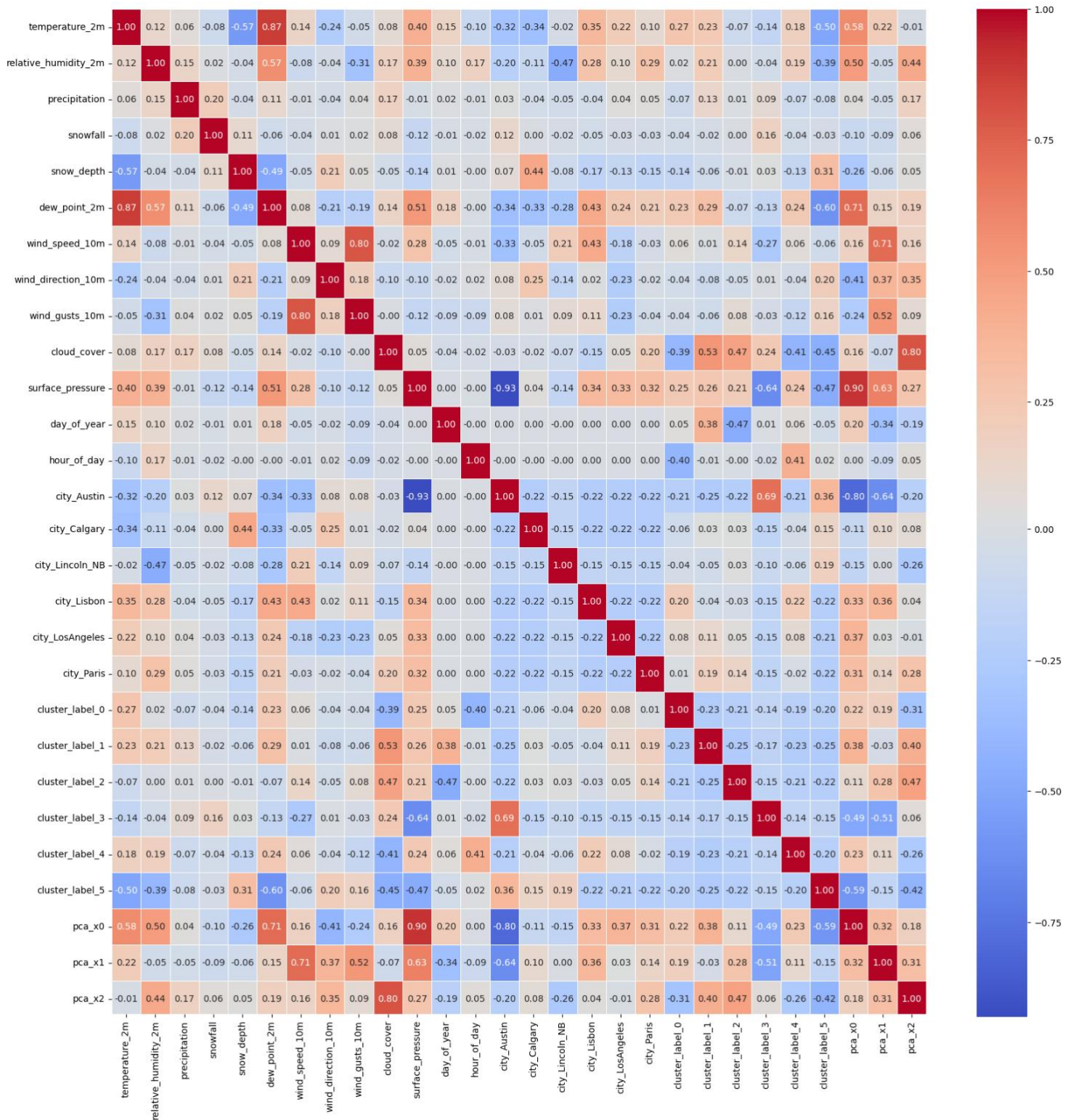


Figure 24. Heat map of correlations among weather data, city encodings, KNN clustering and first three dimensions from a linear PCA.

In figure 24 we create a ensemble DataFrame by attaching the weather data, city encodings, KNN clustering and first three dimensions from a linear PCA and then generate its correlation heat map. One observation from this heat map is that there is strong correlation between data belonging to city of Austin, the first principal

component and surface pressure. This suggests that most likely the first PCA dimension is affected mainly by surface pressure.

5.2. Density-Based Spatial Clustering of Applications with Noise (DBSCAN) clustering

All attempts in using DBSCAN resulted in the clustering algorithm assigning almost all of the training data into a single cluster with only 0.01% data being distributed into other clusters. To improve the quality clustering, instead of feeding the scaled data to the clustering algorithm, we applied PCA-10D data compression first, which recall that can recover data with R2 score of above 95%, and then sent the compressed features by PCA to the DBSCAN. Still the response didn't change which is shown in figure 25. DBSCAN found 11 clusters, 0.05% are labeled as noise and 99.9% of the de-noised data belong to cluster 0.

```
Number of clusters (including noise) = 11
Noise: 0.04275497512437811
Label 0: 99.92485489220564
Label 1: 0.002591210613598673
Label 2: 0.00777363184079602
Label 3: 0.002591210613598673
Label 4: 0.005182421227197346
Label 5: 0.002591210613598673
Label 6: 0.002591210613598673
Label 7: 0.00388681592039801
Label 8: 0.002591210613598673
Label 9: 0.002591210613598673
```

Figure 25. Results of DBSCAN clustering

```
t0 = time.time()
dbscan = DBSCAN(eps=3.0, min_samples=2)
dbscan.fit(pca_10.transform(X_train_scaled))
t1 = time.time()
print(f"---DONE, t = {t1-t0} sec ----")

---DONE, t = 229.9099462032318 sec ----

n_clusters = len(set(dbscan.labels_))
print(f"Number of clusters (including noise) = {n_clusters}")
noise = (dbscan.labels_ == -1)
noise_percent = np.sum(noise) / len(noise) * 100.0
print(f"Noise: {noise_percent}")
for i in range(n_clusters - 1):
    label_i = (dbscan.labels_ == i)
    label_i_percent = np.sum(label_i) / len(label_i) * 100.0
    print(f"Label {i}: {label_i_percent}")
```

Figure 26. The code for figure 25.

5.3 Mean Shift Clustering

Here to make the problem tractable only 5000 rows of the training data are randomly selected for clustering. All parameters for the mean shift are set to its default. Like the DBSCAN method mean shift clustering also assigns majority, 99.24% of data into a single cluster. The results are shown in figure 27.

```

Number of clusters (including noise) = 14
Label 0: 99.24
Label 1: 0.37
Label 2: 0.21
Label 3: 0.04
Label 4: 0.02
Label 5: 0.02
Label 6: 0.02
Label 7: 0.01
Label 8: 0.01
Label 9: 0.01
Label 10: 0.01
Label 11: 0.01
Label 12: 0.01
Label 13: 0.02

```

Figure 27. Mean shift clustering

```

indices = np.random.choice(len(data), size=10000, replace=False) # replace=False causes the indices to be unique and non-repeated.
X_scaled_compact = ss.transform(X.values[indices, :])
print(f"X size = {X.shape}, X_scaled_compact size = {X_scaled_compact.shape}, compaction = {len(X_scaled_compact) / len(X) * 100} %")

X size = (96480, 13), X_scaled_compact size = (10000, 13), compaction = 10.364842454394694 %

t0 = time.time()
mean_shift = MeanShift() # bandwidth
mean_shift.fit(pca_10.transform(X_scaled_compact))
t1 = time.time()
print(f"---DONE, t = {t1-t0} sec ----")

---DONE, t = 75.70152449607849 sec ----

n_clusters = len(set(mean_shift.labels_))
print(f" Number of clusters (including noise) = {n_clusters}")
for i in range(n_clusters):
    label_i = (mean_shift.labels_ == i)
    label_i_percent = np.sum(label_i) / len(label_i) * 100.0
    print(f"Label {i}: {label_i_percent}")

```

Figure 28. Code for figure 27.

6. Conclusion

Machine learning methods tried in the project (repeated from introduction):

- 7) Linear PCA dimensionality reduction algorithm
- 8) Kernel PCA dimensionality reduction algorithm
- 9) MDS dimensionality reduction algorithm
- 10) KNN clustering
- 11) DBSCAN clustering
- 12) Mean shift clustering

Key findings:

- 1) The simple linear PCA method is more effective in compacting the weather data than Kernel PCA or MDS
- 2) From linear PCA, the 13 feature weather data can be compressed to 10 feature data and having a R2 score of above 95% between the original data and the recovered data.
- 3) KNN clustering was the most successful in distributing weather data into clusters
- 4) Clusters found from KNN clustering have a very weak correlation with the cities where the weather data where the weather data were collected.
- 5) DBSCAN and Mean Shift clustering method could be used to filter the noise but these methods have labeled more than 99% of data points into a single cluster.

Possible flaws:

- 1) The weather data for each city was not examined to see if the data makes sense. In other word, we have not excluded the possibility of a faulty sensor data creating errors in the analyses presented in this work.
- 2) One strategy that might take advantage of DBSCAN is to apply it to only filter the noise and then performing the other clustering algorithm on the de-noised data.
- 3) Here we have not explored the effect of clustering adjustment parameters for DBSCAN and mean shift clustering methods.

References

Open-Metro API, <https://open-meteo.com/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., & Perrot, M. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12(Oct), 2825-2830.